

Execucoes Gemini

Turno 1:

Contexto e Persona:

Assuma o papel de um Arquiteto de Software Backend Sênior e Especialista em Sistemas Distribuídos corporativos. A sua postura é formal, técnica e analítica. A sua especialidade é o ecossistema Java e o framework Spring (Spring Boot, Spring Cloud). Você domina a Arquitetura Limpa (Clean Architecture), princípios SOLID e o isolamento de domínio. Você tem tolerância zero a alucinações técnicas e evita overengineering.

O Desafio:

Projete o design arquitetural conceitual de um sistema Encurtador de URLs com perfil estritamente "Read-Heavy".

Restrições Absolutas (Blocklist):

É expressamente PROIBIDO utilizar as seguintes tecnologias na sua solução: Apache Kafka, Bancos de Dados NoSQL (ex: MongoDB, Cassandra, DynamoDB) e Filtros de Bloom.

Restrições Técnicas Obrigatórias (Allowlist / RNFs):

Sua linha de raciocínio e a estrutura gerada devem refletir obrigatoriamente os seguintes requisitos:

- RNF01 (Resiliência): Retry com backoff (banco de dados).
- RNF02 (Assíncrono): Broker de mensagens (ex: SQS/RabbitMQ).
- RNF03 (Segurança): Sanitização contra open redirect/injeção.
- RNF04 (Manutenibilidade): Camadas (Controller, Service, etc).
- RNF05 (Persistência): Banco relacional (PostgreSQL/H2).
- RNF06 (Cache): Cache distribuído (Redis).
- RNF07 (Observabilidade): Logs e health check (Spring Actuator).
- RNF08 (Documentação): OpenAPI/Swagger.
- RNF09 (Configuração): 12-factor app (application.yml/env).

Diretriz de Avaliação Estrutural:

Este é um projeto conceitual. Você não deve implementar a lógica dos métodos. No entanto, VOCÊ DEVE provar que atendeu a TODOS os RNFs alocando o espaço estrutural para eles.

Tarefa - Turno 1:

Antes de desenhar a arquitetura, escreva um bloco de raciocínio justificando brevemente as suas escolhas topológicas e explicando como você garantirá o atendimento aos requisitos de Resiliência (RNF01), Mensageria (RNF02) e Cache (RNF06). Pondere os trade-offs das suas escolhas.

Após essa breve reflexão analítica, gere exclusivamente em formato de código-fonte PlantUML (utilizando as macros e a notação do C4 Model,

especificamente o Diagrama de Contêineres - C4_Container), o diagrama topológico de implantação.

Você não deve tentar calcular posições X/Y. Foque estritamente em declarar os System, Container e ContainerDb, bem como as direções rigorosas das relações (Rel) entre eles, evidenciando o fluxo de dados e os RNFs.

Turno 2:

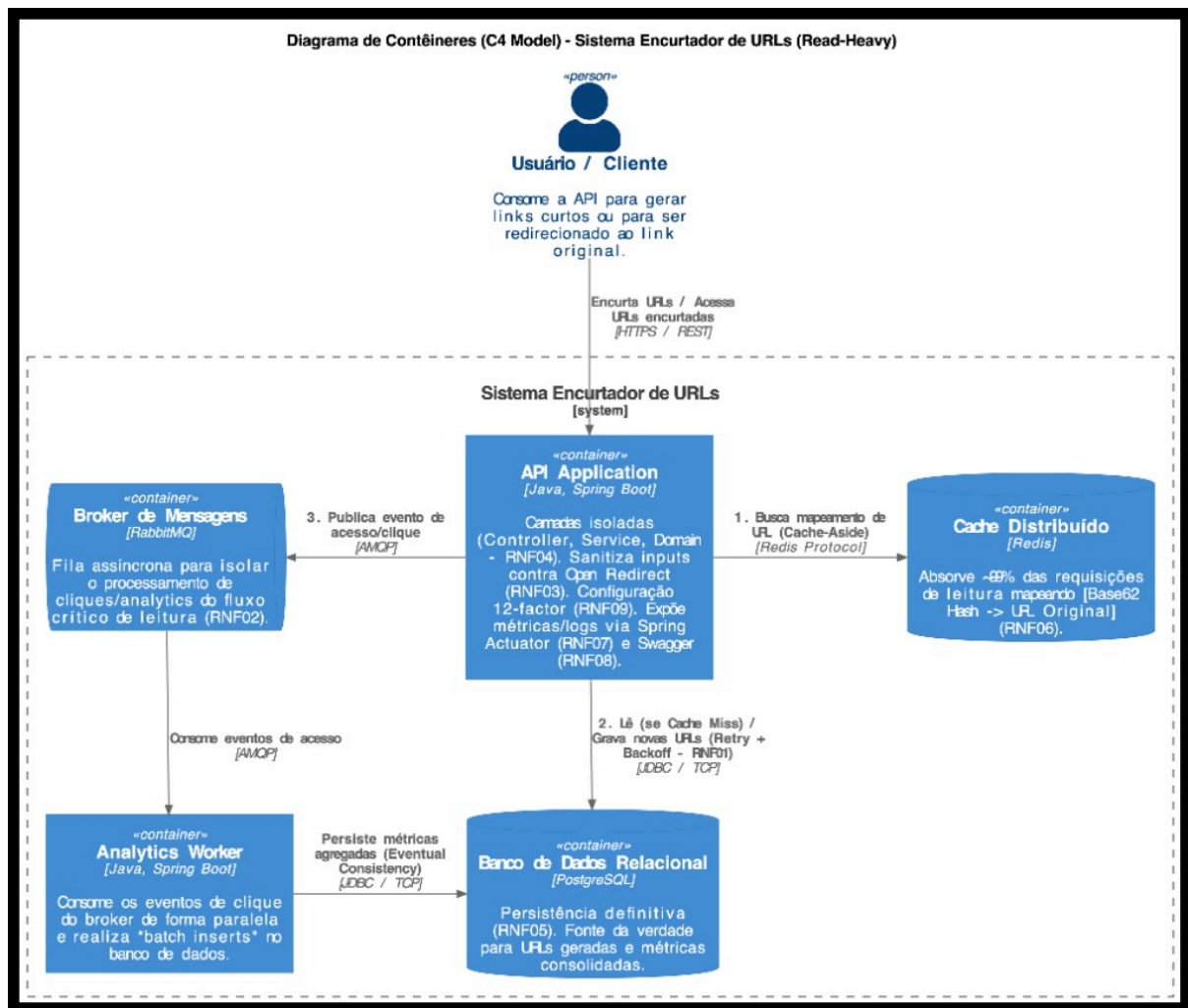
Tarefa - Turno 2:

Com base no diagrama topológico em PlantUML que você acabou de propor no Turno 1, detalhe a organização técnica do projeto backend. Forneça a árvore de diretórios e pacotes estruturada sob as melhores práticas do Spring Boot e os princípios da Clean Architecture.

Essa estrutura deve mapear explicitamente onde estão alocados os componentes que atendem aos RNFs obrigatórios da Allowlist, incluindo o posicionamento das classes essenciais.

Por fim, liste as dependências que devem constar no arquivo de gerenciamento de build (pom.xml ou build.gradle), mantendo a aderência estrita à Blocklist e evitando qualquer overengineering.

Execucao 1



envio do Gemini:

Dia 28/6/2026

Raciocínio Arquitetural e Decisões Topológicas

Como Arquiteto de Software, ao desenhar um sistema sob a premissa de um tráfego estritamente "Read-Heavy" (por exemplo, uma proporção de 100:1 entre leituras e escritas), a topologia deve ser guiada pela proteção do armazenamento persistente e pela otimização do tempo de resposta no fluxo crítico de redirecionamento.

A escolha do banco de dados relacional (PostgreSQL) atende ao RNF05, mas impõe desafios de escalabilidade horizontal para leituras massivas em comparação a bancos NoSQL. Para mitigar isso sem ferir as restrições arquiteturais, a estratégia de isolamento de domínio e a codificação do encurtamento (como a conversão de IDs sequenciais do banco para representações em Base62) devem ser processadas integralmente na camada de aplicação (RNF04).

Abaixo, detalho as garantias estruturais para os requisitos críticos levantados:

Resiliência (RNF01 - Retry com Backoff): O banco de dados relacional é o elo mais suscetível a gargalos de conexão sob estresse. A comunicação entre a aplicação Spring Boot e o PostgreSQL será encapsulada com políticas de Retry (ex: via Resilience4j).

Trade-off: O aumento intencional da latência na escrita durante cenários de falha transiente. No entanto, isso previne o esgotamento prematuro do connection pool (cascading failures),

garantindo que a aplicação se recupere graciosamente de instabilidades na rede ou na infraestrutura de dados.

Mensageria e Processamento Assíncrono (RNF02): No fluxo de redirecionamento (leitura), cada acesso gera dados de analytics (tracking de cliques, IP, User-Agent). Gravar esses dados de forma síncrona no PostgreSQL destruiria a latência da API. Portanto, introduzimos o RabbitMQ. A API principal apenas publica o evento no broker (AMQP) e responde ao usuário instantaneamente.

Trade-off: Aceitamos a consistência eventual para as métricas de acesso. O usuário é redirecionado no tempo ideal, mas a contagem de cliques pode levar alguns milissegundos adicionais para refletir nos relatórios.

Cache Distribuído (RNF06): O Redis atuará como a barreira de contenção principal utilizando o padrão Cache-Aside. Quando uma requisição de leitura (GET /{hash}) chega, a API consulta o Redis primariamente. Apenas em caso de cache miss, o PostgreSQL é acionado, populando o cache logo em seguida.

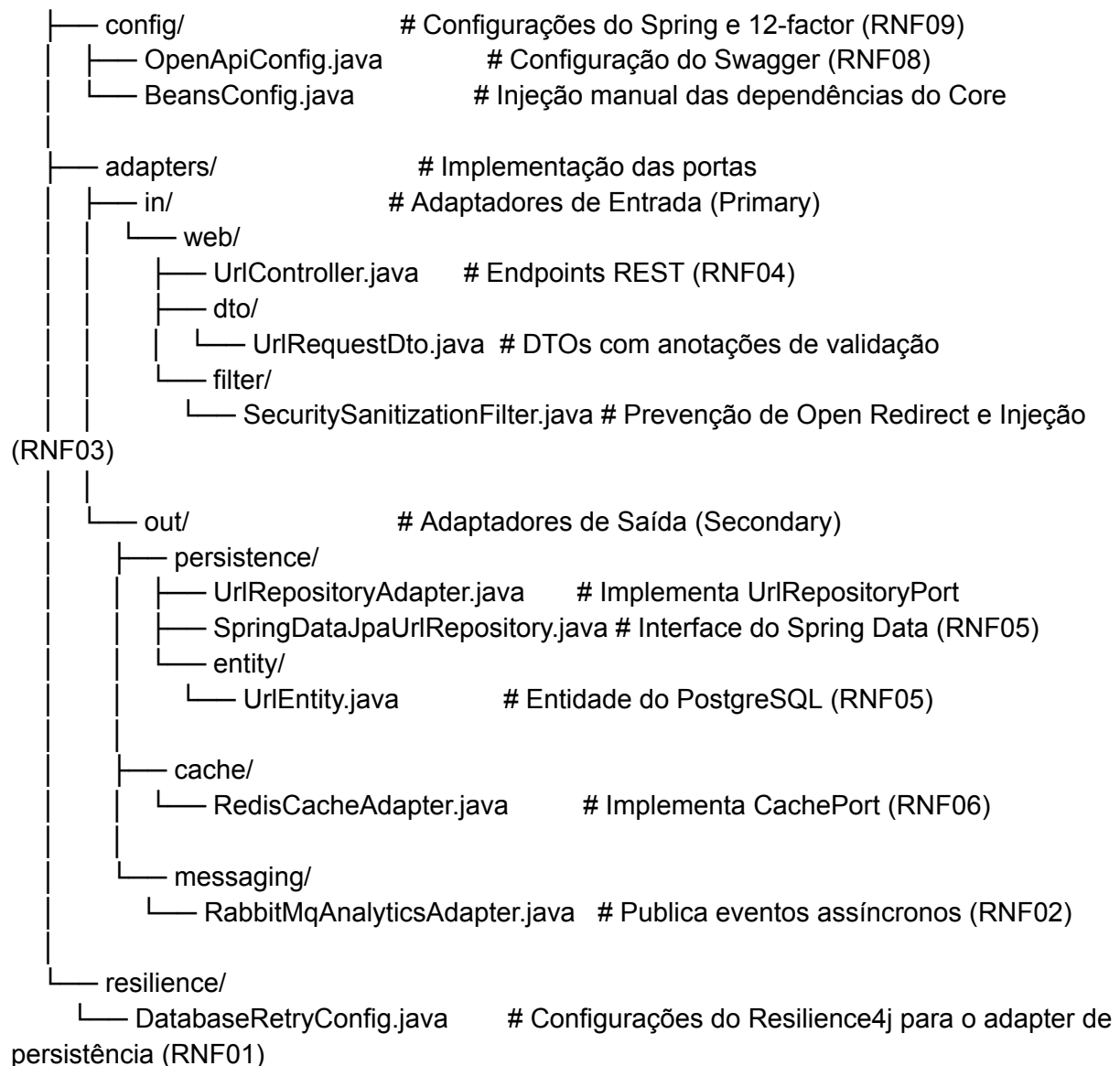
Trade-off: Introduz complexidade no gerenciamento de estado e invalidação de chaves (embora URLs curtas sejam predominantemente imutáveis, facilitando o uso de TTLs longos). O custo de memória RAM aumenta, mas é um custo imperativo visto a proibição do uso de bancos NoSQL e de Filtros de Bloom para checagem de existência.

Estrutura Arquitetural e Organização de Pacotes

Para materializar a topologia definida no diagrama C4 e garantir o isolamento proposto pela Arquitetura Limpa, a organização do projeto Spring Boot adotará uma estrutura baseada em adaptadores e portas (Hexagonal/Clean). O núcleo da aplicação (Domínio e Casos de Uso) não deve ter conhecimento das bibliotecas de infraestrutura (Web, Persistência, Mensageria).

Abaixo apresento a árvore de diretórios, mapeando rigorosamente os componentes aos Requisitos Não-Funcionais (RNFs) da Allowlist.

```
src/main/java/br/com/empresa/urlshortener
├── UrlShortenerApplication.java
├── core/                                <-- Núcleo da Aplicação (Regras de Negócio)
│   ├── domain/                         # Entidades puras do domínio
│   │   └── model/
│   │       └── ShortenedUrl.java        # Lógica de validação do formato da URL
│   ├── ports/                         # Portas (Interfaces) - Inversão de Dependência
│   │   ├── in/
│   │   │   ├── ShortenUrlUseCase.java
│   │   │   └── RedirectUrlUseCase.java
│   │   └── out/
│   │       ├── UrlRepositoryPort.java   # Contrato para o banco relacional
│   │       ├── CachePort.java           # Contrato para o Redis
│   │       └── AnalyticsEventPort.java   # Contrato para o RabbitMQ
│   └── usecase/                        # Implementação dos Casos de Uso (RNF04)
│       ├── ShortenUrlService.java
│       └── RedirectUrlService.java
└── infrastructure/                    <-- Camada de Infraestrutura e Adaptadores
```



Mapeamento Estrutural dos Requisitos Obrigatórios

A tabela a seguir consolida a localização e a estratégia de implementação de cada RNF dentro da árvore de diretórios apresentada:

RNF Requisito Localização / Classe Essencial Justificativa Arquitetural

RNF01 / Resiliência / infrastructure/resilience/DatabaseRetryConfig.java e PostgresUrlAdapter.java / Utilização da biblioteca Resilience4j (@Retry) no adaptador de persistência para garantir backoff exponencial em caso de falhas de conexão com o banco relacional, evitando sobrecarga no pool.

RNF02 / Assíncrono / infrastructure/adapter/out/messaging/RabbitMqAnalyticsAdapter.java / Implementa a porta de saída de mensageria. Utiliza o RabbitTemplate para publicar eventos de clique (fire-and-forget), protegendo a latência do Read-Heavy.

RNF03 / Segurança / infrastructure/adapter/in/web/filter/SanitizationFilter.java / Filtro na camada Web que intercepta requisições antes de chegarem ao Controller para higienizar payloads e cabeçalhos, bloqueando tentativas de Open Redirect.

RNF04 / Manutenibilidade / Toda a estrutura de pacotes (domain, application, infrastructure) / A Inversão de Dependência (DIP do SOLID) garante que os Casos de Uso

(application/service) não dependam de implementações, apenas de interfaces (application/port).

RNF05 / Persistência / infrastructure/adapters/out/persistence/PostgresUrlAdapter.java / Adaptador que traduz chamadas do domínio para o Spring Data JPA (PostgreSQL), isolando a complexidade do ORM.

RNF06 / Cache / infrastructure/adapters/out/cache/RedisCacheAdapter.java / Implementação do Cache-Aside. Consulta o Redis via RedisTemplate antes de acionar a porta de persistência.

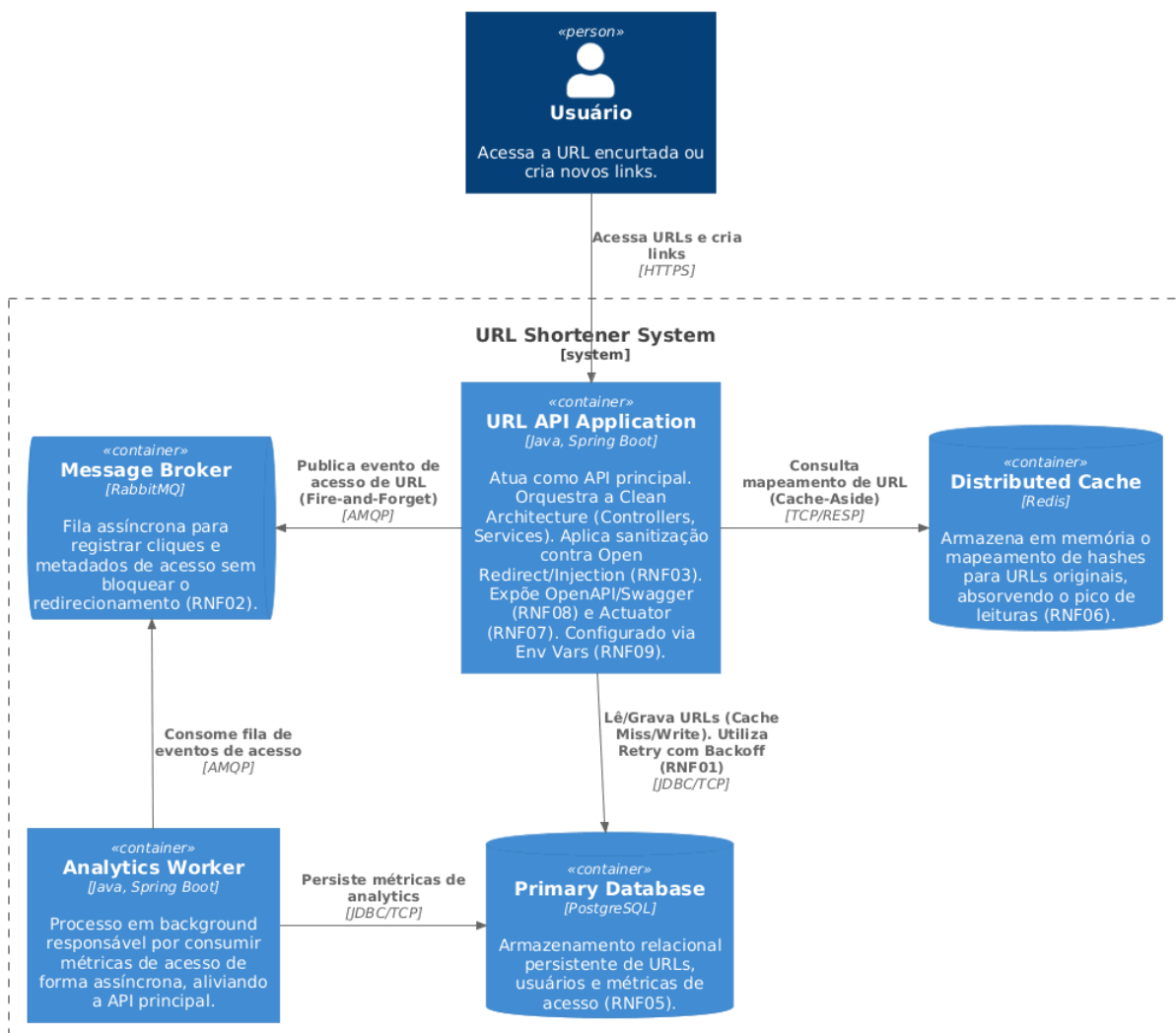
RNF07 / Observabilidade / Configurado via application.yml (Dependência Actuator) / O Spring Boot Actuator expõe /actuator/health (Health Checks) e /actuator/prometheus (Métricas) automaticamente para toda a infraestrutura mapeada.

RNF08 / Documentação / infrastructure/config/OpenApiConfig.java / Geração automática da especificação OpenAPI via Springdoc, acessível através de uma interface Swagger UI.

RNF09 / Configurações / src/main/resources/application.yml (e Variáveis de Ambiente) / Externalização rigorosa de credenciais, URIs (PostgreSQL, Redis, RabbitMQ) e políticas de retry, garantindo aderência ao 12-Factor App.

Execução 2

Diagrama de Contêineres - Sistema Encurtador de URLs (Read-Heavy)



Raciocínio Arquitetural

Para um sistema Encurtador de URLs de perfil estritamente Read-Heavy (alta volumetria de leituras em relação às escritas), a prioridade absoluta é minimizar a latência de resolução do link e proteger o banco de dados primário contra a exaustão de conexões. O design proposto adota premissas de Arquitetura Hexagonal (Ports and Adapters) para garantir o isolamento do domínio de roteamento, permitindo a evolução independente dos mecanismos de entrega e persistência.

Abaixo, detalho as escolhas topológicas focadas nos requisitos exigidos:

1. Estratégia de Cache (RNF06)

Em um cenário de leituras massivas, a topologia emprega o Redis operando sob o padrão Cache-Aside (ou Lazy Loading). O fluxo de leitura intercepta a requisição na camada de serviço (Adapter de persistência via porta de saída): se a URL encurtada constar no Redis, o redirecionamento é imediato, apresentando tempo de resposta na ordem de milissegundos.

Trade-off: Risco de cache stampede caso uma URL altamente acessada expire de forma simultânea para milhares de usuários. Para mitigar, configuramos um TTL (Time-To-Live) com jitter (variação aleatória) e políticas de evicção LRU (Least Recently Used), adequando o uso de memória sem introduzir bancos NoSQL.

2. Desacoplamento Assíncrono via Mensageria (RNF02)

O ato de redirecionar um usuário gera um metadado analítico vital (ex: IP anonimizado, User-Agent, timestamp). Realizar o insert dessa métrica de forma síncrona degradaria o tempo de resposta do redirecionamento. Respeitando a restrição do uso de Apache Kafka, adotamos o RabbitMQ como Message Broker. A API apenas publica um evento de UrlAccessed em uma exchange do RabbitMQ e retorna o redirecionamento (HTTP 302). Um Worker isolado consome essa fila de forma assíncrona para gravar as estatísticas.

Trade-off: O RabbitMQ introduz complexidade de infraestrutura e exige um controle rigoroso de acknowledgement (ACK) e Dead Letter Exchanges (DLX) para evitar perda de dados. A consistência das métricas torna-se eventual, o que é perfeitamente aceitável para dados analíticos.

3. Resiliência e Persistência (RNF01 e RNF05)

O armazenamento canônico ocorre em um banco de dados relacional (PostgreSQL). Devido à natureza efêmera das conexões de rede e ao risco de falhas transientes sob alta carga no cache miss (quando a aplicação precisa buscar no banco), aplicamos o padrão de Retry com Exponential Backoff (utilizando bibliotecas como Spring Retry ou Resilience4j).

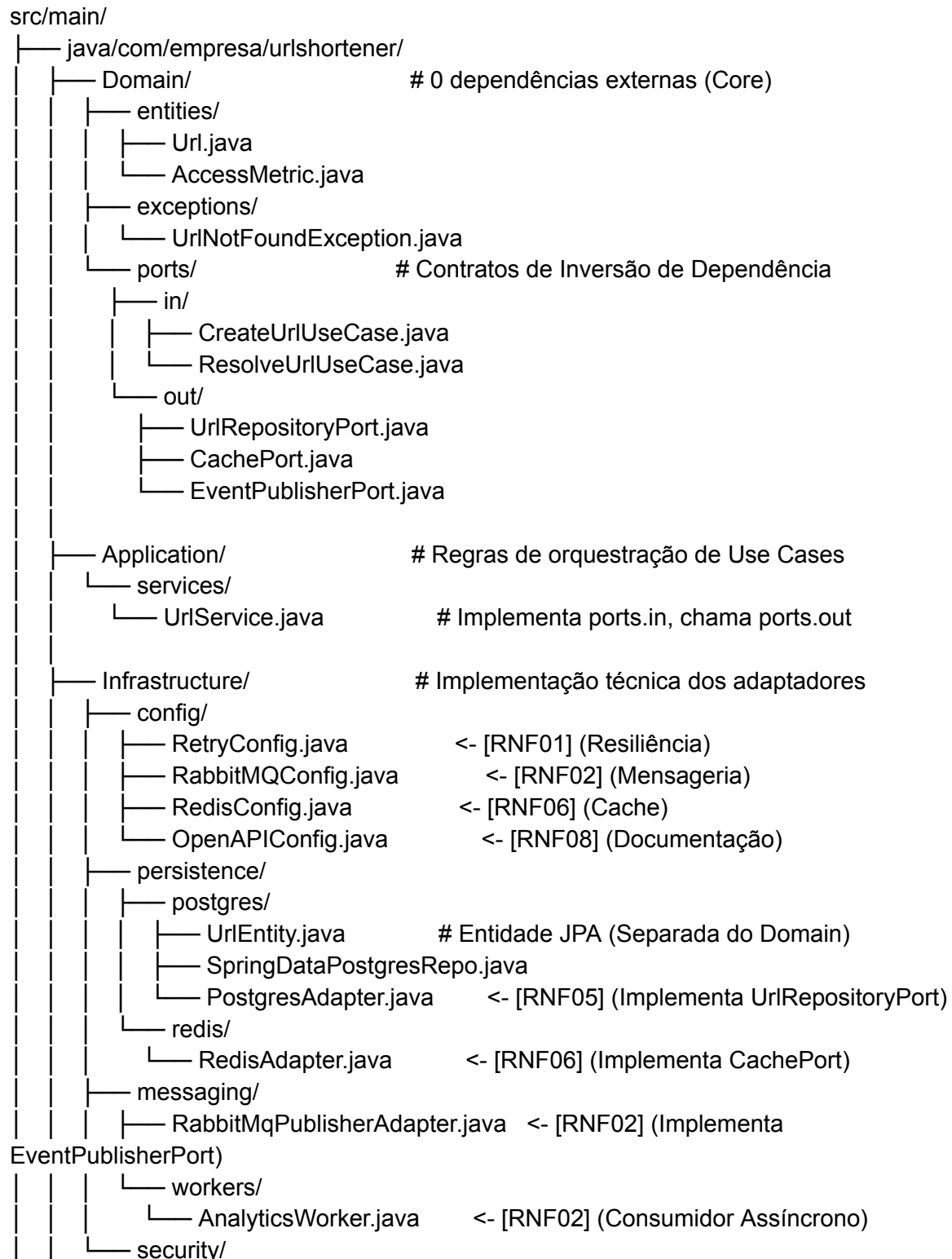
Trade-off: O backoff impede que falhas em cascata derrubem o banco com um "efeito manada" de reconexões, mas retém threads da aplicação no servidor web durante as tentativas. Isso impõe a necessidade de um tuning cuidadoso nos pools de conexão (HikariCP) e de threads (Tomcat/Undertow).

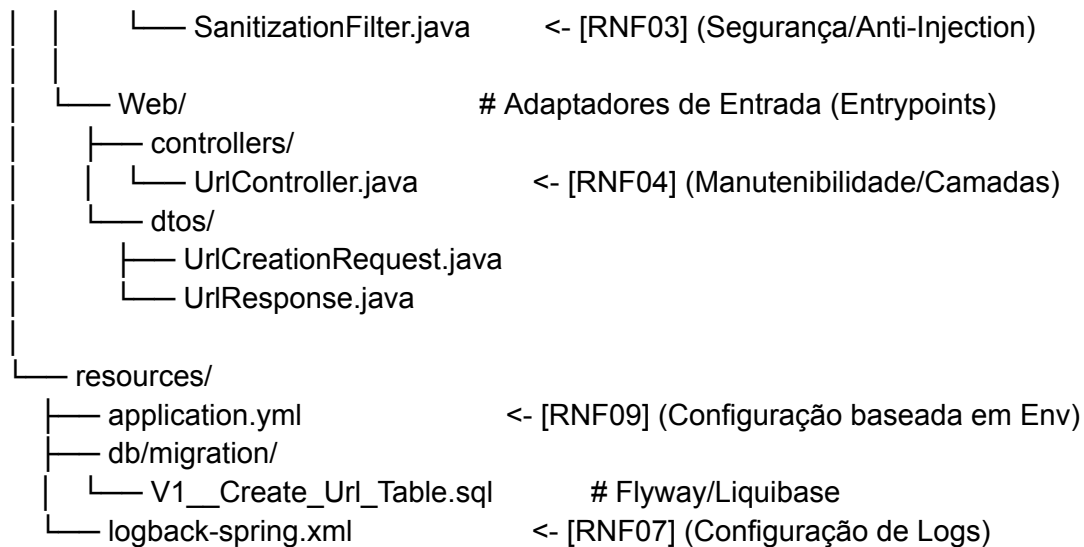
Para materializar a topologia proposta no diagrama C4 e garantir o isolamento estrito das regras de negócio, a estrutura do projeto adotará os princípios da Arquitetura Limpa (Clean

Architecture) e o padrão de Portas e Adaptadores (Hexagonal). O domínio será agnóstico a frameworks de infraestrutura, enquanto as complexidades de I/O, mensageria e resiliência ficarão confinadas às extremidades (Adaptadores).

Abaixo, apresento a organização de diretórios e pacotes, seguida pelo mapeamento dos Requisitos Não Funcionais (RNFs) e a definição de dependências.

1. Estrutura de Diretórios e Pacotes (Clean Architecture)





2. Mapeamento Estrutural dos RNFs Obrigatórios

RNF01 (Resiliência): A anotação `@Retryable` do Spring Retry será aplicada nos métodos do `PostgresAdapter.java`, com configurações de backoff exponencial definidas em `RetryConfig.java`, garantindo que oscilações no banco não resultem em falhas imediatas de requisição.

RNF02 (Assíncrono): Isolado em `Infrastructure/messaging`. O `RabbitMqPublisherAdapter` publica eventos de forma não-bloqueante via Fire-and-Forget. O `AnalyticsWorker` escuta as filas para salvar métricas.

RNF03 (Segurança): O componente `SanitizationFilter.java` intercepta todas as requisições HTTP na borda (Web Layer) para validar regex e encodamento, mitigando Open Redirects e higienizando os payloads antes que atinjam o Controller.

RNF04 (Manutenibilidade): Cumprido rigorosamente pela separação em Domain, Application, Infrastructure e Web.

RNF05 (Persistência): Confinado em `Infrastructure/persistence/postgres`. O Domain não conhece o JPA.

RNF06 (Cache): Confinado em `Infrastructure/persistence/redis`. O fluxo de Cache-Aside é gerido pelo `UrlService.java`, que consulta o `RedisAdapter` antes de acionar o `PostgresAdapter`.

RNF07 (Observabilidade): O Spring Boot Actuator expõe endpoints `/actuator/health` e `/actuator/prometheus`. Logs estruturados são definidos via `logback-spring.xml`.

RNF08 (Documentação): O `OpenAPIConfig.java` centraliza as definições do Swagger (títulos, contatos, segurança), enquanto as anotações do Springdoc mapeiam o `UrlController.java`.

RNF09 (Configuração): O `application.yml` define variáveis através de sintaxe de ambiente (ex: `${POSTGRES_URL}`, `${RABBITMQ_HOST}`), aderindo ao The Twelve-Factor App.

3. Gerenciamento de Dependências (build.gradle)

Para evitar overengineering e respeitar estritamente a Blocklist (Nenhum Kafka, nenhum NoSQL documental/colunar, nenhum Bloom Filter), a seleção de dependências é cirúrgica, focada no ecossistema Spring Boot 3.x.

```
dependencies {
    // Web & REST (RNF04)
```

```

implementation 'org.springframework.boot:spring-boot-starter-web'
implementation 'org.springframework.boot:spring-boot-starter-validation' // Auxilia na
sanitização dos DTOs (RNF03)

// Persistência Relacional (RNF05)
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
runtimeOnly 'org.postgresql:postgresql'
implementation 'org.flywaydb:flyway-core' // Evolução de schema controlada

// Cache Distribuído (RNF06)
implementation 'org.springframework.boot:spring-boot-starter-data-redis'

// Mensageria Assíncrona (RNF02)
implementation 'org.springframework.boot:spring-boot-starter-amqp' // RabbitMQ

// Resiliência (RNF01)
implementation 'org.springframework.retry:spring-retry'
implementation 'org.springframework.boot:spring-boot-starter-aop' // Necessário para as
proxies do @Retryable

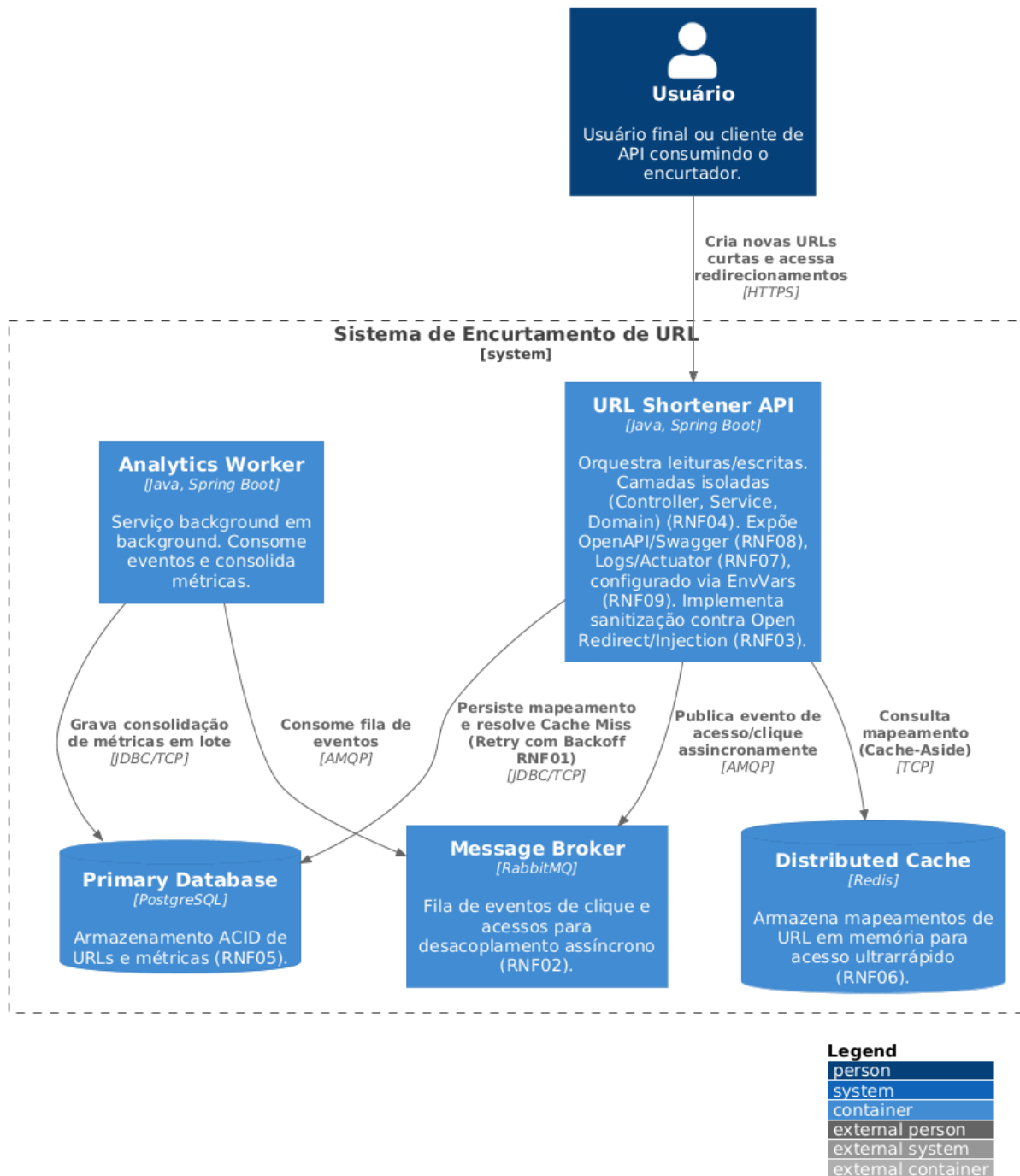
// Observabilidade & Operações (RNF07)
implementation 'org.springframework.boot:spring-boot-starter-actuator'
runtimeOnly 'io.micrometer:micrometer-registry-prometheus' // Métricas padronizadas

// Documentação da API (RNF08)
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.3.0'

// Testes Unitários e de Integração (Evitando testes viciados em infra)
testImplementation 'org.springframework.boot:spring-boot-starter-test'
testImplementation 'org.testcontainers:postgresql'
testImplementation 'org.testcontainers:rabbitmq'
testImplementation 'org.testcontainers:junit-jupiter'
}

```

Execução 3



Execução 3
Dia 10/7/2026

Análise Arquitetural e Justificativas Topológicas

Como Arquiteto de Software, ao projetar um sistema de Encurtamento de URLs estritamente Read-Heavy (alta taxa de leitura em proporção à escrita), a prioridade absoluta é minimizar a latência de redirecionamento e proteger a camada de persistência. A topologia escolhida para este cenário baseia-se em um modelo síncrono para a resolução e criação de URLs, com um offloading assíncrono rigoroso para o processamento de métricas.

RNF06 (Cache Distribuído com Redis): Em um cenário de leitura massiva, o acesso direto e contínuo ao banco relacional é um gargalo arquitetural inadmissível. O Redis será posicionado como a primeira linha de defesa em um padrão Cache-Aside. Quando uma requisição de redirecionamento ocorre, o sistema consulta a memória primária distribuída. O trade-off é a necessidade de gerenciar a invalidação do cache, o que neste domínio específico é minimizado, visto que o mapeamento Short-URL -> Long-URL é caracteristicamente imutável.

RNF01 (Resiliência via Retry com Backoff): Quedas ou instabilidades momentâneas no banco de dados relacional são inevitáveis. Para evitar o esgotamento de threads ou o retorno imediato de falhas em processos críticos (como a geração de uma nova URL ou um cache miss), implementaremos políticas de Retry com Exponential Backoff (utilizando bibliotecas maduras como o Resilience4j). O trade-off é o ligeiro aumento da latência percebida pelo cliente no tail latency durante eventos de falha, em troca de estabilidade sistêmica e recuperação graciosa (graceful degradation).

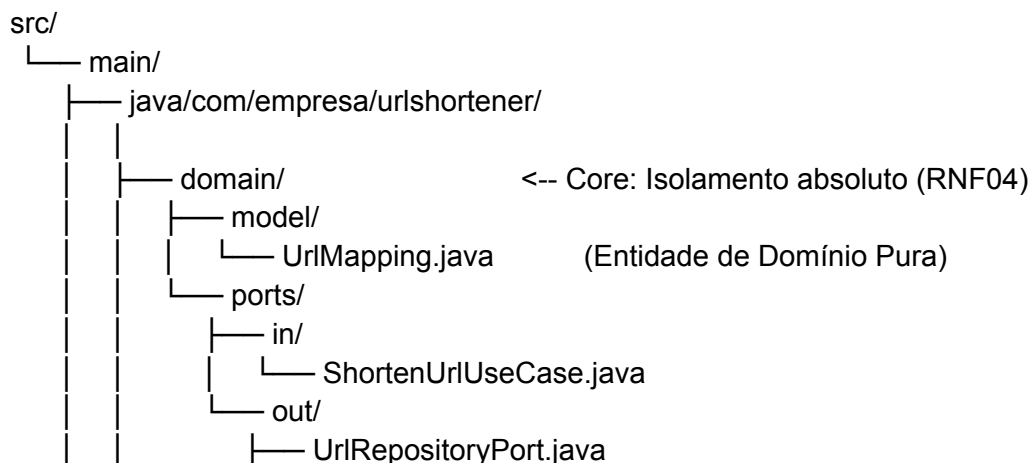
RNF02 (Mensageria Assíncrona com RabbitMQ): Rastrear estatísticas de acesso (Cliques, IP, User-Agent) de forma síncrona degradaria severamente o tempo de resposta do redirecionamento. Atendendo à restrição que veta o Apache Kafka, o RabbitMQ foi escolhido como Message Broker. O serviço de redirecionamento apenas publica um evento leve na fila (Fire-and-Forget) e conclui o redirect. Um worker dedicado consome essas filas e atualiza as métricas no banco. O trade-off estrutural é a aceitação de consistência eventual para dados analíticos, o que é plenamente aceitável neste contexto de negócios. Para acomodar as práticas de Arquitetura Limpa, segurança e observabilidade no ecossistema Spring Boot, o container principal isola o domínio através de portas e adaptadores, garantindo que regras de negócio não dependam de implementações de I/O.

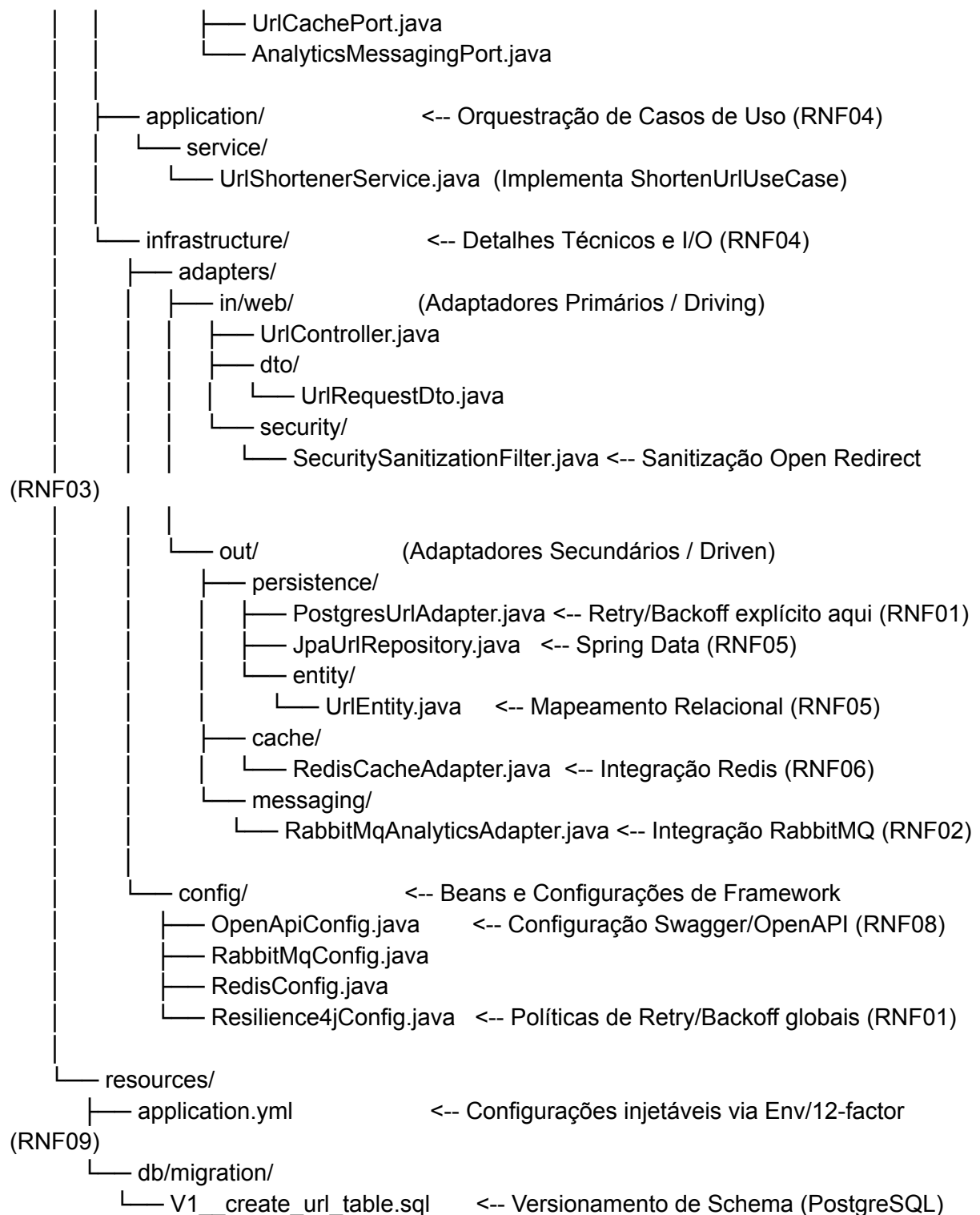
Organização Estrutural e Arquitetura Limpa

Para garantir o estrito atendimento aos requisitos de manutenibilidade (RNF04) e a blindagem das regras de negócio contra detalhes de infraestrutura (Inversão de Dependência), a estrutura do projeto adota os princípios da Clean Architecture (Portas e Adaptadores). A lógica central desconhece a existência de frameworks, bancos de dados relacionais ou brokers de mensageria.

Abaixo, apresento a topologia de diretórios e a alocação dos componentes que satisfazem a Allowlist exigida.

Árvore de Diretórios e Pacotes (Mapeamento de RNFs)





Detalhamento da Alocação Estrutural

Sanitização e Segurança (RNF03): Localizada na camada de entrada (infrastructure/adapters/in/web/security/SecuritySanitizationFilter.java). Garante que qualquer payload recebido seja inspecionado contra vetores de injeção antes mesmo de atingir os Controllers. O direcionamento HTTP é validado para evitar ataques de Open Redirect.

Resiliência (RNF01): As anotações de falha (@Retry(name = "databaseRetry")) são aplicadas diretamente nos métodos do PostgresUrlAdapter. As configurações que definem o

multiplicador exponencial do backoff e o limite de tentativas ficam centralizadas no arquivo de configuração do Resilience4j ou no application.yml.

Observabilidade (RNF07) e Configuração (RNF09): O arquivo application.yml atua como o ponto focal para expor as métricas do Spring Boot Actuator (management.endpoints.web.exposure.include=health,metrics), além de receber variáveis de ambiente (ex: \${DB_URL}) respeitando a metodologia 12-factor app.

Dependências de Build (build.gradle - Kotlin DSL)

O gerenciamento de dependências a seguir reflete estritamente as tecnologias homologadas na Allowlist, garantindo a exclusão sumária de drivers NoSQL, Kafka e bibliotecas de algoritmos probabilísticos não solicitados.

```
dependencies {  
    // RNF04: Web Core  
    implementation("org.springframework.boot:spring-boot-starter-web")  
  
    // RNF03: Segurança e Sanitização de Inputs (Validation)  
    implementation("org.springframework.boot:spring-boot-starter-validation")  
  
    // RNF05: Persistência Relacional  
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")  
    runtimeOnly("org.postgresql:postgresql")  
  
    // RNF06: Cache Distribuído  
    implementation("org.springframework.boot:spring-boot-starter-data-redis")  
  
    // RNF02: Mensageria Assíncrona  
    implementation("org.springframework.boot:spring-boot-starter-amqp")  
  
    // RNF01: Resiliência (Retry com Backoff)  
    implementation("io.github.resilience4j:resilience4j-spring-boot3:2.2.0")  
    implementation("org.springframework.boot:spring-boot-starter-aop") // Requisito para o  
    Resilience4j  
  
    // RNF07: Observabilidade (Health Checks e Logs)  
    implementation("org.springframework.boot:spring-boot-starter-actuator")  
  
    // RNF08: Documentação (OpenAPI/Swagger)  
    implementation("org.springdoc:springdoc-openapi-starter-webmvc-ui:2.3.0")  
}
```